

Proposta de Hackathon

Processamento Inteligente de Documentos (IDP) com Amazon Bedrock

Mentores: Layrane Dantas, Lucas Leite, Gilson Castro

1. Caso de Uso

A fintech CrediFácil processa centenas de solicitações de empréstimo com garantia imobiliária por dia. Cada solicitação chega como um pacote de 5 a 8 documentos em PDF, identidade, comprovantes de renda, extratos bancários e documentação do imóvel, que precisam ser lidos, classificados e digitados manualmente por operadores humanos.

1.1. Por que isso é um problema de negócio?

- Perda de receita - Clientes desistem da solicitação durante a espera
- Custo operacional - Equipe grande dedicada apenas à digitação
- Erros geram retrabalho - CPF errado, endereço incompleto, valor de renda incorreto → reproprocessamento
- Não escala - Em campanhas promocionais ou datas de pico, o gargalo é humano
- Risco regulatório - Erros em dados de clientes podem gerar problemas de compliance

2. O Desafio

Construir uma solução de IDP (Intelligent Document Processing) que recebe o pacote de documentos de empréstimo, classifica cada tipo de documento, extrai os dados e os organiza em um JSON estruturado e padronizado.

2.1. O que se espera da solução?

Uma pipeline automatizada que:

- Recebe o pacote de documentos.

- Classifica automaticamente cada tipo de documento.
- Extrai os dados relevantes de cada um.
- Retorna um JSON padronizado, completo e confiável.
- Funciona ponta a ponta, do upload ao resultado final.

2.2. Pré-requisitos

- Conceitos básicos de S3, Lambda e IAM
- Noções de Python (boto3)
- Conta AWS com acesso ao Amazon Bedrock Data Automation (BDA) e ao Amazon Nova na região us-east-1

3. Base de Dados

<https://github.com/aws-samples/sample-document-processing-with-amazon-bedrock-data-automation/tree/main/20-Industry-Use-Cases/21-Mortgage-and-Lending/documents>

4. Serviços e Competências

Esses são serviços recomendados para desenvolver a solução.

Competência	Serviço AWS
Classificação e extração de documentos	Amazon Bedrock Data Automation
Estruturação confiável de dados (JSON)	Amazon Nova (Bedrock) tool calling
Armazenamento de documentos e resultados	Amazon S3
Automação do fluxo	AWS Lambda, AWS Step Functions e Amazon EventBridge
Permissões e segurança	IAM

5. Entregáveis

- Projeto BDA extraindo os dados do pacote de documentos
- JSON estruturado e padronizado gerado com apoio do Amazon Nova
- Pipeline automatizado (Lambda + S3) ligando upload → extração → estruturação
- Apresentação de 15 minutos demonstrando o fluxo

- Implementação Front-End (opcional/ bônus)
- Mapeamento da organização (opcional/ bônus)
- A estruturação dos dados em planilha do Excel (opcional/bônus)

6. Limites Técnicos (Ferramentas e Frameworks)

Para garantir que as soluções fiquem alinhadas à expectativa e ao foco em IDP na AWS:

Permitido / Obrigatório:

- Uso obrigatório do Amazon Bedrock Data Automation (BDA) para extração
- Uso do Amazon Nova (Bedrock) para estruturação dos dados
- Serviços AWS: S3, Lambda, IAM
- Linguagem Python (boto3)
- Bibliotecas auxiliares simples (ex: pandas, streamlit para o dashboard bônus)

Não permitido:

- Soluções que não usem o BDA (ex: OCR próprio, parsers manuais)
- Serviços de IA de outros provedores (fora da AWS)
- Inserir dados fora dos documentos fornecidos
- Bibliotecas/frameworks que substituam o papel do BDA ou do Bedrock

7. Estação de Mentoria

Estação	Mentor	Foco
BDA	Gilson	Blueprints, projetos, extração
Bedrock	Layrane/Gilson	Modelos LLM
Serverless	Lucas/Layrane	Lambda, S3, automação
IAM / Setup	Lucas	Permissões e configuração de ambiente

A equipe deve procurar o mentor certo conforme dúvidas.

8. Materiais de Apoio

- Amazon Nova: [Structured outputs with Amazon Nova: A guide for builders | Artificial Intelligence](#)
- AWS Lambda: [Documentação do AWS Lambda](#)
- Amazon Bedrock-BDA: [Como funciona a Automação de Dados do Bedrock - Amazon Bedrock](#)

- Amazon Bedrock: [Amazon Bedrock Documentation](#)
- Amazon S3: [Documentação do Amazon Simple Storage Service](#)
- AWS IAM: [Documentação do AWS Identity and Access Management](#)

9. Dica para o time

- Comecem testando a extração de um documento individual antes de montar o pipeline completo.
- Definam o schema JSON cedo, quanto antes definirem a estrutura de saída, mais fácil conectar as peças.
- Tool calling é seu aliado. Não tente fazer parsing de texto livre gerado pelo modelo.
- Testem iterativamente documento por documento, depois o pacote completo.
- JSON é o produto final, foquem na qualidade do JSON. É isso que será avaliado.